

DevOps Interview Preparation

Bash & Python Scripting — Complete Q&A Guide

Tailored for junior DevOps engineers with internship experience in AWS, Kubernetes, Docker, Jenkins, and Terraform.

1. Shell Scripting Fundamentals

Q: What is shell scripting?

Shell scripting is writing a series of commands in a text file that the shell (command interpreter) executes one by one. It lets you automate repetitive tasks instead of running commands manually every time.

Q: Why do DevOps engineers use shell scripts?

- Automate deployments, backups, and monitoring tasks.
- Reduce human error in repetitive operations.
- Schedule tasks using cron jobs.
- Quickly glue together Linux commands into workflows.

Q: What is Bash?

Bash (Bourne Again Shell) is the most widely used shell on Linux systems. It interprets and executes shell scripts and is the default shell on most Linux servers.

Q: What is a shebang (#!/bin/bash)?

The shebang is the very first line of a script. It tells the operating system which interpreter to use to run the script.

```
#!/bin/bash
# This line means: use /bin/bash to run this script
```

Q: How do you execute a shell script?

- Make it executable: `chmod +x script.sh`
- Run it: `./script.sh`
- Or run directly: `bash script.sh`

Q: What are shell variables? Difference between local and environment variables?

- Shell variable: stores a value within the script — e.g., `NAME="Rahul"`
- Local variable: only available inside the current script or function.
- Environment variable: available to the current shell AND all child processes. Created using `export` — e.g., `export PATH`.

Q: What are positional parameters? What is \$??

- `$1, $2, $3 ...` → command-line arguments passed to the script.
- `$#` → total number of arguments.

- •\$@ → all arguments as separate words.
- •\$? → exit status of the last command (0 = success, non-zero = failure).

Q: Explain this script: `#!/bin/bash echo "Hello World"`

Line 1: Shebang — tells the OS to use Bash as the interpreter. Line 2: Prints the text Hello World to the terminal. This is the simplest possible Bash script.

2. Bash Variables & Parameters

Q: How do you declare variables and read user input?

```
NAME="Rahul"           # assign a variable
read -p "Enter name: " NAME # read from user input
```

Q: What are \$*, \$@, and \$# ?

- •\$# → count of arguments passed.
- •\$@ → all arguments; each treated as a separate string (safer in loops).
- •\$* → all arguments as one single string (splits on spaces).
- •Rule of thumb: always use \$@ in loops to avoid issues with spaces.

Q: What is command substitution?

Command substitution captures the output of a command and stores it in a variable.

```
TODAY=$(date)
echo "Today is $TODAY"
```

Q: Difference between single and double quotes?

- •Double quotes "" → variables and commands inside are expanded/evaluated.
- •Single quotes ' ' → everything inside is treated as literal text, no expansion.

```
NAME="Rahul"
echo "Hello $NAME" # Output: Hello Rahul
echo 'Hello $NAME' # Output: Hello $NAME
```

3. Conditional Statements

Q: Difference between [(single bracket) and [[(double bracket)?

- •[is the POSIX-compatible test command. Works in any shell.
- •[[is Bash-specific, more powerful — supports regex matching, handles empty variables more safely.
- •Recommendation: use [[]] in Bash scripts.

Q: How do you compare strings and integers?

```
# String comparison
if [[ "$NAME" == "Rahul" ]]; then echo "Match"; fi

# Integer comparison
if [ "$COUNT" -gt 5 ]; then echo "Greater than 5"; fi

# Operators: -eq, -ne, -gt, -lt, -ge, -le
```

Q: How do you check if a file or directory exists?

```
if [ -f file.txt ]; then echo "File exists"; fi
if [ -d /logs ]; then echo "Directory exists"; fi
if [ -e path ]; then echo "Path exists (file or dir)"; fi
```

Q: What is exit status and how do you check command success?

- Every command returns an exit code when it finishes.
- 0 = success. Non-zero = failure.
- \$? holds the exit status of the last command.

```
cp file.txt /backup/  
if [ $? -eq 0 ]; then  
    echo "Copy successful"  
else  
    echo "Copy failed"  
fi
```

Q: Explain: if [-f file.txt]; then echo "File exists"; fi

-f checks if the path is a regular file (not a directory). If true, it prints 'File exists'. The fi keyword ends the if block.

Q: What is a case statement?

A case statement is a cleaner alternative to multiple if-elif blocks, especially when checking one variable against many values.

```
case $ENV in  
    dev) echo "Development" ;;  
    prod) echo "Production" ;;  
    *) echo "Unknown" ;;  
esac
```

4. Loops

Q: What is a for loop vs while loop?

- for loop: used when you know the number of iterations in advance (looping over a list, files, range).
- while loop: used when you loop until a condition becomes false (unknown number of iterations).

```
# For loop  
for SERVER in web1 web2 web3; do  
    echo "Checking $SERVER"  
done  
  
# While loop  
COUNT=1  
while [ $COUNT -le 5 ]; do  
    echo "Iteration $COUNT"  
    ((COUNT++))  
done
```

Q: How do you break a loop or skip an iteration?

- break → exits the entire loop immediately.
- continue → skips the current iteration and moves to the next.

Q: How do you iterate through files in a directory?

```
for FILE in /var/log/*.log; do  
    echo "Processing $FILE"  
done
```

Q: Common DevOps use cases for loops?

- Checking disk usage or service status on multiple servers.
- Processing log files in a directory.
- Deploying the same config to multiple environments.
- Retrying a failed command a set number of times.

5. Functions

Q: What are shell functions and why use them?

Functions are reusable blocks of code inside a script. They help avoid repetition, make scripts easier to read, and allow you to test individual parts separately.

Q: How do you define, call, and pass arguments to a function?

```
check_service() {  
    SERVICE=$1          # first argument  
    systemctl is-active --quiet $SERVICE  
    if [ $? -eq 0 ]; then  
        echo "$SERVICE is running"  
    else  
        echo "$SERVICE is DOWN"  
    fi  
}  
  
# Call the function  
check_service nginx  
check_service docker
```

Q: How do functions return values?

- Bash functions return an exit code (0-255) using return.
- To return a string/value, use echo inside the function and capture it with command substitution.

```
get_date() {  
    echo $(date +%Y-%m-%d)  
}  
  
TODAY=$(get_date)  
echo "Date: $TODAY"
```

6. Text Processing Commands

Q: What is grep and how is it used?

grep searches for a pattern inside a file or output. It is one of the most-used commands in DevOps for log analysis.

```
grep "ERROR" app.log          # find lines with ERROR  
grep -i "error" app.log       # case-insensitive  
grep -r "timeout" /var/log/   # search recursively  
grep -c "ERROR" app.log       # count matching lines  
grep -v "DEBUG" app.log       # show lines NOT matching
```

Q: What is awk and how do you use it?

awk is a text processing tool that works column by column. Perfect for extracting fields from structured logs or CSV files.

```
# Print 1st column (default delimiter: space)
awk '{print $1}' access.log

# Print 2nd column using comma as delimiter
awk -F',' '{print $2}' data.csv

# NR = current line number; FS = field separator
awk 'NR > 1 {print NR, $1}' file.txt
```

Q: What is sed and how do you replace text?

sed is a stream editor — it reads input line by line and applies transformations like find-and-replace. Very useful in CI/CD pipelines to update config files.

```
sed 's/dev/prod/g' config.txt      # replace all 'dev' with 'prod'
sed -i 's/dev/prod/g' config.txt   # edit the file in-place
sed '5d' file.txt                  # delete line 5
```

Q: What are cut, sort, uniq, tr, and xargs?

- cut: extracts specific columns — e.g., `cut -d' ' -f1 /etc/passwd` (get usernames)
- sort: sorts lines alphabetically or numerically — `sort -n file.txt`
- uniq: removes duplicate adjacent lines (use after sort) — `sort file.txt | uniq -c`
- tr: translates or deletes characters — `echo 'hello' | tr 'a-z' 'A-Z' → HELLO`
- xargs: takes output from one command and passes it as arguments to another — `find . -name '*.log' | xargs rm`

7. Linux Automation & Cron Jobs

Q: What is cron and crontab?

Cron is the Linux job scheduler. It runs commands or scripts automatically at specified times. crontab is the file where you define these scheduled jobs.

```
# crontab format:
# MIN HOUR DAY MONTH WEEKDAY COMMAND

0 2 * * * /scripts/backup.sh      # Run at 2:00 AM daily
*/5 * * * * /scripts/health.sh    # Run every 5 minutes
0 0 * * 0 /scripts/cleanup.sh     # Run every Sunday midnight
```

Q: Script works manually but fails in cron — why?

- Cron runs with a minimal environment — `$PATH` is very limited.
- Fix: use full absolute paths in your script (e.g., `/usr/bin/python3`, `/bin/bash`).
- Also check: file permissions, and redirect output to a log for debugging.

```
# Add this to crontab to capture errors:
0 2 * * * /scripts/backup.sh >> /var/log/backup.log 2>&1
```

Q: How would you automate a disk usage check and alert?

```
#!/bin/bash
THRESHOLD=80
USAGE=$(df / | awk 'NR==2 {print $5}' | tr -d '%')
```

```
if [ "$USAGE" -gt "$THRESHOLD" ]; then
    echo "ALERT: Disk usage is ${USAGE}%" | mail -s "Disk Alert" admin@company.com
fi
```

8. Python Fundamentals for DevOps

Q: Why is Python popular in DevOps?

- Easy to read and write compared to Bash for complex logic.
- Rich library ecosystem: boto3 (AWS), requests (HTTP), paramiko (SSH).
- Better error handling, data structures, and object-oriented support.
- Works cross-platform (Linux, Windows, macOS).

Q: When would you choose Python over Bash?

- Complex logic, multi-step workflows, or data processing tasks.
- When you need to interact with APIs (AWS, Kubernetes, Slack).
- When handling JSON/YAML configuration files.
- Bash is better for quick one-liners and simple file/system operations.

Q: What are lists, dictionaries, tuples, and sets?

```
# List: ordered, mutable
servers = ["web1", "web2", "db1"]

# Dictionary: key-value pairs
config = {"host": "localhost", "port": 5432}

# Tuple: ordered, immutable
coords = (10, 20)

# Set: unordered, no duplicates
unique_ips = {"10.0.0.1", "10.0.0.2"}
```

9. Python Scripting Questions

Q: How do you read and write a file in Python?

```
# Read a file
with open("app.log") as f:
    for line in f:
        print(line.strip())

# Write to a file
with open("output.txt", "w") as f:
    f.write("Deployment complete\n")
```

The with statement ensures the file is automatically closed after the block, even if an error occurs.

Q: What is try-except and why is it important?

try-except handles errors gracefully so your script doesn't crash unexpectedly. Critical for production automation scripts.

```
try:
    with open("config.json") as f:
        data = json.load(f)
except FileNotFoundError:
    print("Config file not found")
except json.JSONDecodeError:
    print("Invalid JSON format")
```

Q: How do you parse JSON in Python?

```
import json

# Parse JSON string
data = json.loads('{"name": "Rahul", "role": "DevOps"}')
print(data["name"])    # Output: Rahul

# Read JSON from a file
with open("config.json") as f:
    config = json.load(f)

# Write JSON to a file
with open("output.json", "w") as f:
    json.dump(config, f, indent=2)
```

Q: What is argparse and when would you use it?

argparse lets you accept command-line arguments in Python scripts, making them flexible and reusable without editing the code.

```
import argparse

parser = argparse.ArgumentParser(description="Backup tool")
parser.add_argument("--env", required=True, help="Environment: dev or prod")
parser.add_argument("--dry-run", action="store_true")
args = parser.parse_args()

print(f"Running for: {args.env}")
```

10. Python Automation for DevOps

Q: What is Boto3 and how do you authenticate with AWS?

Boto3 is the official AWS SDK for Python. It lets you manage AWS services programmatically — EC2, S3, IAM, CloudWatch, etc.

```
import boto3

# Authentication: boto3 uses credentials from:
# 1. Environment variables (AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY)
# 2. ~/.aws/credentials file
# 3. IAM role attached to the EC2/Lambda instance (recommended in production)

s3 = boto3.client("s3", region_name="ap-south-1")
ec2 = boto3.resource("ec2", region_name="ap-south-1")
```

Q: How do you list S3 buckets, upload a file, and create an EC2 instance?

```
import boto3

# List S3 buckets
s3 = boto3.client("s3")
response = s3.list_buckets()
for bucket in response["Buckets"]:
    print(bucket["Name"])

# Upload file to S3
s3.upload_file("app.log", "my-bucket", "logs/app.log")

# Create EC2 instance
ec2 = boto3.resource("ec2")
ec2.create_instances(
    ImageId="ami-0abcdef1234567890",
    MinCount=1, MaxCount=1,
    InstanceType="t2.micro",
    KeyName="my-key"
)
```

Q: How would you automate log analysis in Python?

```
import re
from collections import Counter

error_counts = Counter()
with open("app.log") as f:
    for line in f:
        if "ERROR" in line:
            # Extract error type using regex
            match = re.search(r"ERROR.*?(\\w+Exception)", line)
            if match:
                error_counts[match.group(1)] += 1

for error, count in error_counts.most_common(5):
    print(f"{error}: {count} times")
```

11. Python & Kubernetes Questions

Q: How would you interact with Kubernetes using Python?

Use the official Kubernetes Python client library (kubernetes). It lets you list Pods, check restarts, restart deployments, and more — all programmatically.

```
from kubernetes import client, config

# Load kubeconfig (from ~/.kube/config or in-cluster)
config.load_kube_config()

v1 = client.CoreV1Api()

# List all Pods in 'default' namespace
```



```

pods = v1.list_namespaced_pod(namespace="default")
for pod in pods.items:
    restarts = pod.status.container_statuses[0].restart_count
    print(f"{pod.metadata.name} – restarts: {restarts}")

```

Q: How would you automatically restart a deployment?

```

from kubernetes import client, config
import datetime

config.load_kube_config()
apps_v1 = client.AppsV1Api()

# Trigger a rollout restart by patching an annotation
patch = {
    "spec": {
        "template": {
            "metadata": {
                "annotations": {
                    "kubectrl.kubernetes.io/restartedAt":
                        datetime.datetime.utcnow().isoformat()
                }
            }
        }
    }
}

apps_v1.patch_namespaced_deployment("my-app", "default", patch)
print("Deployment restarted")

```

12. Python Coding Questions

Q: Reverse a string

```

s = "DevOps"
print(s[::-1])    # Output: spOveD

```

Q: Find duplicate elements in a list

```

items = ["nginx", "docker", "nginx", "k8s", "docker"]
duplicates = [x for x in set(items) if items.count(x) > 1]
print(duplicates)    # ['nginx', 'docker']

```

Q: Count word frequency in a log file

```

from collections import Counter

with open("app.log") as f:
    words = f.read().split()

freq = Counter(words)
print(freq.most_common(5))

```

Q: Check if a string is a palindrome

```
def is_palindrome(s):
    s = s.lower().replace(" ", "")
    return s == s[::-1]

print(is_palindrome("racecar")) # True
print(is_palindrome("devops"))  # False
```

Q: Monitor disk usage and generate an alert

```
import shutil

def check_disk(path="/", threshold=80):
    total, used, free = shutil.disk_usage(path)
    usage_pct = (used / total) * 100
    if usage_pct > threshold:
        print(f"ALERT: Disk usage at {usage_pct:.1f}% on {path}")
    else:
        print(f"Disk usage OK: {usage_pct:.1f}%")

check_disk("/")
```

Q: Find the largest file in a directory

```
import os

def largest_file(directory):
    files = [(f, os.path.getsize(os.path.join(directory, f)))
             for f in os.listdir(directory)
             if os.path.isfile(os.path.join(directory, f))]
    largest = max(files, key=lambda x: x[1])
    print(f"Largest: {largest[0]} ({largest[1]} bytes)")

largest_file("/var/log")
```

13. Real-World DevOps Scenarios

Q: Scenario 1: Check disk usage across 100 servers

Use SSH in a loop with a list of servers. Python with paramiko or Bash with ssh in a for loop.

```
#!/bin/bash
SERVERS=("web1" "web2" "db1" "db2") # or read from file
for SERVER in "${SERVERS[@]}; do
    echo "=== $SERVER ==="
    ssh -o StrictHostKeyChecking=no $SERVER "df -h /"
done
```

Q: Scenario 2: Automatically restart a failed service

Use a Bash script with systemctl and schedule it via cron.

```
#!/bin/bash
SERVICE="nginx"
if ! systemctl is-active --quiet $SERVICE; then
    echo "$(date): $SERVICE is down. Restarting..." >> /var/log/service_monitor.log
    systemctl restart $SERVICE
```

```
fi
```

Q: Scenario 3: Parse a 10GB log file efficiently

- Never load the entire file into memory — process it line by line.
- Use grep or awk for simple searches (fastest — they're written in C).
- In Python, use a generator (open + for loop) to stream line by line.

```
# Python – memory-efficient
with open("huge.log") as f:
    for line in f:          # reads one line at a time
        if "ERROR" in line:
            print(line.strip())
```

Q: Scenario 4: Create AWS resources daily

Write a Python Boto3 script and schedule it as a cron job or Lambda function triggered by EventBridge.

Q: Scenario 5: Identify Pods restarting frequently

```
from kubernetes import client, config

config.load_kube_config()
v1 = client.CoreV1Api()
pods = v1.list_pod_for_all_namespaces()

for pod in pods.items:
    for cs in (pod.status.container_statuses or []):
        if cs.restart_count > 5:
            print(f"{pod.metadata.namespace}/{pod.metadata.name} "
                  f"— restarts: {cs.restart_count}")
```

Q: Scenario 6: Validate environment before Jenkins deployment

```
#!/bin/bash
# Pre-deployment validation script
ERRORS=0

# Check required env vars
for VAR in APP_ENV DEPLOY_KEY DB_HOST; do
    if [ -z "${!VAR}" ]; then
        echo "ERROR: $VAR is not set"
        ((ERRORS++))
    fi
done

# Check service connectivity
if ! curl -sf http://db-service:5432/health > /dev/null; then
    echo "ERROR: Cannot reach database"
    ((ERRORS++))
fi

[ $ERRORS -eq 0 ] && echo "Validation passed" || exit 1
```

14. Troubleshooting Questions

Q: Script works manually but fails in cron

- Problem: cron has a minimal \$PATH — it doesn't know where commands like python3 or aws are.
- Fix: Use full absolute paths in your script: /usr/bin/python3, /usr/local/bin/aws.
- Fix: Add PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin at the top of your crontab.
- Debug: Redirect cron output to a file: * * * * * /script.sh >> /tmp/cron.log 2>&1

Q: Variable values are empty inside the script

- Check quotes: unquoted variables can break with spaces.
- Use set -x at the top of your script to enable debug mode and print each line.
- Check if the variable is set: [-z "\$VAR"] && echo 'VAR is empty'

Q: Python: ModuleNotFoundError

- The module isn't installed in the Python environment being used.
- Fix: pip3 install module-name or check which Python is running: which python3.
- In automation: make sure the cron/Jenkins job uses the same virtualenv.

Q: Python script consuming too much memory

- You're probably loading entire files or large data into memory at once.
- Fix: process files line by line (streaming), or use generators.
- For large DataFrames: use pandas chunksize parameter.

15. Behavioral & Experience Questions

Q: How to answer 'What task did you automate during your internship?'

Use the STAR format: Situation → Task → Action → Result. Be specific about what was manual, what you wrote, and the measurable impact.

- Example: 'We were manually restarting failed Docker containers every few hours. I wrote a Bash script that used docker inspect to check container health every 5 minutes via cron, and restarted containers automatically. This eliminated 3-4 manual interventions per day.'
- Always mention: what the pain was, what you used (Bash/Python/cron), and what improved.

Q: How do you test scripts before deploying to production?

- Test on a dev/staging environment first — never production.
- Use bash -n script.sh to syntax-check Bash scripts without running them.
- Add echo statements (dry-run mode) before executing destructive commands.
- For Python: write simple unit tests or run with sample/mock data.
- Always add logging so you can trace what the script did.

Q: How do you ensure scripts are maintainable?

- Add comments explaining the why, not just the what.
- Use functions to organize code into logical sections.
- Store configuration (paths, thresholds) in variables at the top — not hardcoded throughout.
- Use meaningful variable and function names.
- Store scripts in version control (Git).

Focus first on Sections 6 (grep/awk/sed), 7 (cron), 10 (Boto3), and 13 (scenarios) — these come up most often in junior DevOps interviews. Good luck!